VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY

HO CHI MINH UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTER SCIENCE AND ENGINEERING



ASSIGNMENT REPORT (L10)

TOPIC: QA SYSTEM STUDENTFORUM

TEAM: 4

Supervisor: Duong Huynh Anh Duc

Ho Chi Minh, October 2026

TASK ASSIGNMENT AND MEMBER EVALUATION TABLE

The table below presents the detailed task assignments and the evaluation of task completion levels for the members of Group 4.

StudentID	Name	Work	Percentage
2411545	Trần Nguyễn Khánh	- System description - Sketching erd - Compose all related resourse.	100%
2410148	Trần Hồng Anh	- review system description - mapping relations	100%
2411997	Đặng Thị Hoàng Mai	- investigate Reddit platform - fixing errors in the system description - fituning the semantic constraints	100%
2412166	Lê Mai Nam	- StackOverFlow investigation - sketching erd	100%
2411258	Phạm Gia Huy	- complete semantic constraints list - mapping final relations	100%

Ho Chi Minh City, October 2026

TABLE OF CONTENT

TASK ASSIGNMENT AND MEMBER EVALUATION TABLE **1**

1. ASSIGNMENT 1 **3**

1.1 ANALYSE AND DESCRIBE DATA REQUIREMENT 3

1.1.1 Research related systems 3

1.1.2 System description 5

1.1.3 Sematic Constraints 11

1.2 EERD design 12

1.2.1 Entity type and attributes 13

1.2.2 Relations 15

1.3 Mapping from EERD to relational schema 15

PHỤ LỤC **16**

1. ASSIGNMENT 1

1.1 ANALYSE AND DESCRIBE DATA REQUIREMENT

1.1.1 Research related systems

Reference applications: StackOverflow and Reddit platform

Links: Stack Overflow, Reddit

General description about 2 applications: Despite their different cultural orientations, Stack Overflow and Reddit share a common foundational principle: both are community-driven question-and-answer platforms that rely entirely on the power of crowdsourcing. The structure of these systems revolves around a self-moderating content selection mechanism based on voting (upvotes and downvotes), allowing the most useful and high-quality answers to rise to the top while gradually filtering out less valuable contributions. Furthermore, to manage the enormous volume of information generated by their users, both platforms employ effective domain-segmentation mechanisms through tags on Stack Overflow and subreddits on Reddit. These structures help connect users seeking information with communities that possess the relevant expertise. More importantly, the core value of these platforms extends beyond solving immediate problems for individual users. By storing, indexing, and publicly archiving discussions, they create vast knowledge repositories that can be accessed and reused by millions of future users through search engines.

Primary Users of StackOverflow:

- **Question Askers:** Search for solutions, post technical questions with source code, and mark accepted answers.
- **Answer Contributors:** Provide answers, share debugging code snippets, and offer technical explanations.
- **Community Editors:** Vote on content, edit posts to improve clarity, and assign classification tags.
- **Administrators:** Moderate disputes, close duplicate or policy-violating questions, and maintain platform operations.

Basic Business Analysis of StackOverflow:

- **Question Posting and Management:** Create titles, describe technical issues, insert formatted source code, and assign classification tags.
- **Answering and Technical Explanation:** Provide clear solutions, explain algorithms, and suggest optimized approaches.
- **Voting and Answer Acceptance:** The asker marks an accepted answer, while the community upvotes or downvotes content based on quality.
- **Reputation Accumulation:** The system recognizes contributions through Reputation points and badges, granting moderation privileges.
- **Content Moderation:** Review community edits, close duplicate topics, and remove spam.

Primary Users of Reddit:

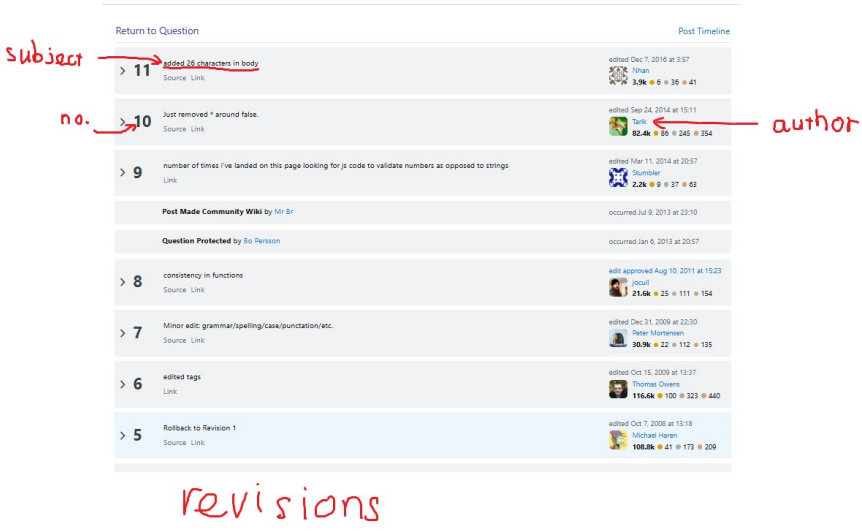
- **Members (Redditors):** Explore feeds, participate in interest-based communities, create posts, and engage in discussions.
- **Community Moderators:** Establish subreddit rules, review posts, lock comments, and manage members.
- **Administrators (Admins):** Operate platform infrastructure, handle content safety, and enforce platform policies.

Basic Business Analysis of Reddit:

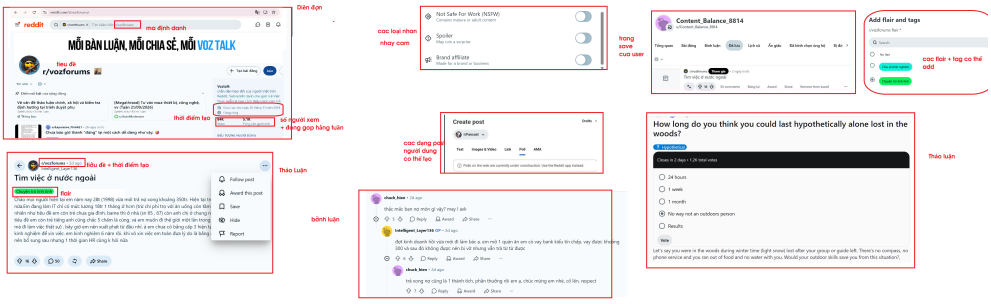
- **Creating and Joining Subreddits:** Join or establish communities based on topics, interests, or areas of expertise.
- **Publishing Multi-format Content:** Share text posts, images, videos, links, or opinion polls.
- **Threaded Discussions:** Support multi-level commenting, enabling detailed interactions within specific conversation branches.

- **Voting and Karma System:** Users upvote or downvote content, while algorithms rank posts according to categories such as Hot, Top, and New.
- **Subreddit Moderation and Administration:** Configure keyword filters, remove posts that violate community rules, and process user reports.

Some interfaces:



Hình 1: *Stack Overflow interface*



Hình 2: *Reddit interface*

1.1.2 System description

Team 4 proposes to develop StudentForum, an online forum system that provides a platform for students from different universities and schools to participate in communities, exchange information, ask questions, share experiences, provide feedback, and communicate with other students. The system is inspired

by community-based discussion platforms such as Reddit and Stack Overflow, while being adapted to the context of student communities.

Each participating university can have its own community, where users can participate according to the requirements established by that community. The system includes students who participate in discussions, community administrators who manage individual communities, a system administrator who manages the overall platform, and students who can be assigned to manage individual forums.

A **USER** represents an account that participates in or manages activities within the StudentForum system. USER is specialized into two overlapping and total subclasses: **STUDENT** and **COMMUNITY_ADMIN**. A STUDENT represents a student who uses the platform to participate in communities and interact with forum content, while a COMMUNITY_ADMIN represents a user responsible for managing a particular community. The system stores common user information such as User_ID, username, password, Email, Date_of_birth, Gender, and Phone. The **Phone** attribute is multivalued because a user may have multiple phone numbers. To ensure that the platform is restricted to students and authorized users, accounts representing students can only log in using a verified educational email address.

A **STUDENT** is a type of USER who participates in StudentForum and interacts with communities and discussion content. The system stores information about a student's program, representing the academic program in which the student is enrolled, such as a standard or high-quality program. Each student studies at exactly one university, while a university can have multiple students, through the STUDY_AT relationship.

A student can become a member of multiple communities, while each community can contain multiple students through the **MEMBER OF** relationship. Through community membership, students can access forums, create posts, write comments, vote on discussion items, and interact with other participants. A student can also propose and create a forum within a community. After the forum creation request is approved by a COMMUNITY_ADMIN, the student can be assigned as a FORUM_ADMIN of that forum through the MANAGE

relationship.

A **COMMUNITY_ADMIN** represents a type of USER responsible for managing a particular community. Through the **CONTROL** relationship, each COMMUNITY_ADMIN manages exactly one COMMUNITY, while a COMMUNITY can be managed by multiple COMMUNITY_ADMINs. COMMUNITY_ADMINs are responsible for managing community activities and reviewing requests related to the creation and management of forums within the community, they have some certain privileges. A COMMUNITY_ADMIN does not necessarily have to be associated with the university represented by the corresponding community. For example, a COMMUNITY_ADMIN responsible for managing the HCMUT Community may be associated with another university. Therefore, the management responsibility of a COMMUNITY_ADMIN is determined by the community to which the administrator is assigned rather than by the university associated with the administrator's own student status.

A **SYSTEM_ADMIN** represents an administrator responsible for managing the StudentForum platform at the system level. The system stores information about the system administrator such as SysadminID and systemAdminName. A SYSTEM_ADMIN can **handle** administrative activities related to universities participating in StudentForum through the HANDLE relationship. The system administrator operates at the platform level and is responsible for managing system-wide activities rather than managing the content of an individual community or forum.

A **UNIVERSITY** represents a university or educational institution participating in StudentForum. Each university is handled by exactly one SYSTEM_ADMIN, while a SYSTEM_ADMIN can handle multiple universities. The system stores information about each university such as Uni_ID, Uni_name, and uni_description. A university can have a corresponding COMMUNITY, while each community corresponds to one university through the CORRESPOND relationship. For example, HUTECH can have a HUTECH Community and HCMUT can have an HCMUT Community. The university therefore provides the institutional context for the corresponding community in StudentForum.

A **COMMUNITY** represents the main discussion space associated with

a particular university. The system stores information about a community such as Com_name, creation_date, and com_description. A community can contain multiple forums through the HAS relationship, while each forum belongs to one community. A community can also have multiple participating students through the **MEMBER** OF relationship and can be managed by multiple COMMUNITY_ADMINs through the CONTROL relationship. The system can derive the number of participants and number of community administrators from these relationships. The community provides the main environment for students to participate in discussions and access smaller forums related to different topics or interests.

A **FORUM** represents a smaller discussion space within a community and focuses on a particular topic or interest. For example, the HUTECH Community may contain forums such as "Street Food of HUTECH" or "Game Net Clubs near HUTECH". A forum stores information such as ForumName, Description, and Creation_date. A community can contain multiple forums, while each forum belongs to one community through the HAS relationship. A student can manage one or more forums through the **MANAGE** relationship. This relationship represents the **FORUM_ADMIN** role. A forum can therefore have multiple FORUM_ADMINs. A FORUM_ADMIN does not necessarily have to be associated with the university corresponding to the forum and is responsible for managing activities and content within the corresponding forum.

Within a forum, students can create **POSTS** to ask questions, share information, discuss topics, provide feedback, or share their experiences and opinions. A post stores information such as Content, created time, and attachment information. Each post is created by one student and belongs to one forum, while a student can create multiple posts and a forum can contain multiple posts. Posts therefore serve as one of the main discussion objects through which students communicate within forums. A student can also modify a post after it has been created, and the system records the corresponding editing history through POST EDIT.

A **TAG** represents a topic or category used to classify posts. The system stores information about a tag such as TagID, Tag Name, and Description.

A post must be associated with at least one tag and may have multiple tags, while a tag can be associated with multiple posts. Therefore, POST and TAG have a many-to-many relationship through the ATTACH relationship. For example, a programming-related post may be associated with tags such as “C++”, “Database”, or “Programming”, while a post about university life may have tags related to courses, campus activities, or student life. Tags allow students to classify and identify posts according to their topics.

A **COMMENT** represents a response or contribution made by a student to a discussion. A student can create multiple comments, while each comment is created by one student. A comment stores information such as Content and created date. Every comment belongs to exactly one post; a comment may additionally reply to another comment of the same post. The recursive PARENT relationship represents this reply structure, where a comment can act as a parent of multiple child comments and a child comment can have a parent comment. This allows StudentForum to support nested discussions in which students can directly respond to previous comments. Students can also modify their comments after creation, and the system records the corresponding editing history through COMMENT EDIT.

A **POST EDIT** represents a record of an editing operation performed on a post. A post can have multiple post-edit records, while each post-edit record belongs to one post through the RECORD relationship. Similarly, a COMMENT EDIT represents a record of an editing operation performed on a comment. A comment can have multiple comment-edit records, while each comment-edit record belongs to one comment through the LOG relationship. These editing records allow the system to maintain information about modifications made to posts and comments instead of only storing their latest versions.

A **DISCUSSIONITEM** represents a generalized discussion object used to associate common interactions with different types of discussion content. The DISCUSSIONITEM abstraction covers POST, COMMENT, and FORUM. This abstraction allows relationships that are applicable to different types of discussion content, such as voting and notification-related activities, to be represented without creating separate relationships for each individual content type. Each

notification is ABOUT at most one discussion item

Users can also subscribe to DISCUSSIONITEMs through the **SUBSCRIBE NOTIFICATION TO** relationship to receive notifications about activities related to discussion content that they are interested in. A user can subscribe to multiple discussion items, while a discussion item can have multiple subscribers. When a user subscribes to a discussion item, the system records the corresponding subscription relationship. When relevant activities occur on a subscribed discussion item, the system can generate a NOTIFICATION for the corresponding user. A user can cancel a subscription at any time, after which the user will no longer receive notifications related to that discussion item.

The **VOTES** relationship represents the voting activity between a STUDENT and a DISCUSSION_ITEM. A student can vote on a discussion item, while a discussion item can receive votes from multiple students. The relationship contains the upvote boolean attribute, which indicates whether the student's current vote is an upvote or a downvote, and the last_vote_date attribute, which records the date and time of the most recent voting action. If a student changes their vote from an upvote to a downvote or vice versa, the system updates the upvote value and last_vote_date of the existing voting relationship. Therefore, the system maintains the student's current voting state for each discussion item rather than treating each change as a separate voting entity.

When displaying comments within a post, the system allows users to sort comments according to creation time, number of upvotes, or number of downvotes. When sorting by upvotes or downvotes, the system calculates the corresponding number of votes associated with each comment from the voting relationship and displays comments in descending order according to the selected vote count. If multiple comments have the same vote count, the newer comment is displayed first. This allows users to identify recent or highly supported comments more easily.

The system also provides **NOTIFICATION** to inform users about activities related to their participation in StudentForum. A user can receive multiple notifications, while each notification belongs to one user through the RECEIVE relationship. A notification stores information such as id, content, noti-date, and

read_status. A notification can also be associated with a DISCUSSION_ITEM through the ABOUT relationship. This allows the system to provide notifications concerning activities related to posts, comments, or forums. For example, a user may receive a notification when another user interacts with discussion content related to them. The read_status attribute indicates whether the recipient has read the notification.

Overall, StudentForum integrates USER, STUDENT, COMMUNITY_ADMIN, SYSTEM_ADMIN, UNIVERSITY, COMMUNITY, FORUM, POST, TAG, COMMENT, POST EDIT, COMMENT EDIT, DISCUSSION_ITEM, and NOTIFICATION, together with their corresponding relationships, to form an integrated student discussion platform. Students can participate in communities associated with different universities, access forums, create and edit posts, classify posts using tags, create and reply to comments, and express their opinions through voting. The DISCUSSION_ITEM abstraction allows common interactions to be applied to posts, comments, and forums. COMMUNITY_ADMIN users manage individual communities, while students assigned through the MANAGE relationship can perform the FORUM_ADMIN role for individual forums. The SYSTEM_ADMIN manages the platform at the system level and handles participating universities and other system-wide activities.

1.1.3 Sematic Constraints

SUMMARIZE ALL THE SEMANTIC CONSTRAINTS:

1. **A STUDENT must log in using a verified educational email address.** The email must be valid and associated with an educational institution participating in StudentForum.
2. A STUDENT can belong to multiple COMMUNITIes, but a student can only be considered a member of a community after satisfying that community's membership requirements.
3. Creation_date of a post cannot be sooner that creation_date of that forum
4. Only a student in the community can request to create a forum in that community

5. No comment can reply to itself: Comment_ID $\neq$ Parent_Comment_ID

6. Parent and child comments must belong to the same post

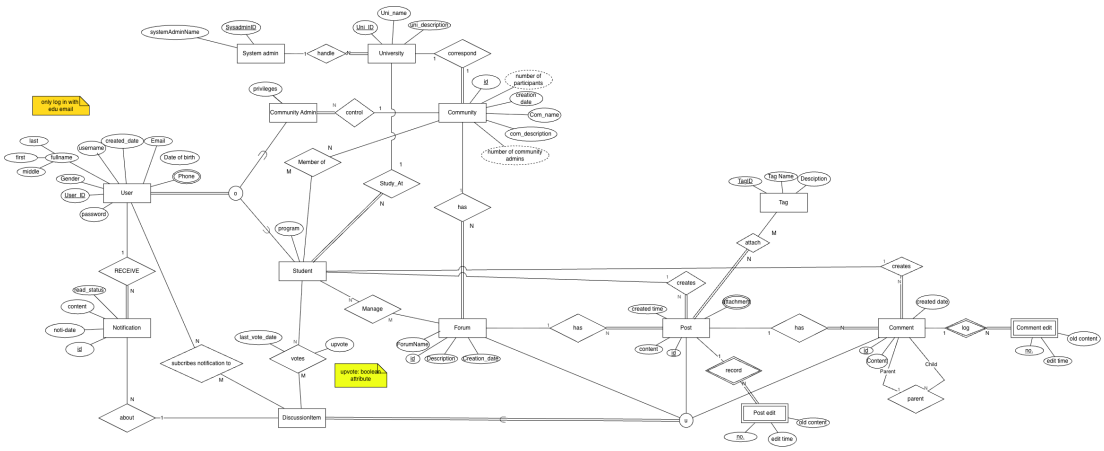
7. A STUDENT may choose not to vote on a DISCUSSION_ITEM. If a student votes on a discussion item, there can be at most one current voting relationship between that STUDENT and DISCUSSION_ITEM. The student can change the vote between an upvote and a downvote, in which case the existing voting relationship is updated. If the student removes their vote, the corresponding voting relationship is deleted.

8. **A POST or COMMENTS can be edited at most 5 times by its creator.** After the fifth edit, the system does not allow another edit.

9. Within the same COMMUNITY, no two FORUMs may have the same ForumName. However, forums belonging to different communities may have the same ForumName.

10. A STUDENT can edit only the POSTs or COMMENTs that they originally created.

1.2 EERD design



Hình 3: *EERD model for StudentForum*

Link for higher resolution: Team 4 eerd

1.2.1 Entity type and attributes

The EERD consists of regular entity types, subtypes of `User`, a category entity type, and two weak entity types. Their attributes are described below. There are 11 strong/regular entity types, 2 weak entity types, 1 category type.
USER (Regular entity):

- `User_ID` (primary key)
- `Fullname` (composite attribute): `First`, `Middle`, `Last`
- `Username`
- `Password`
- `Email`
- `Phone` (multivalued attribute)
- `Gender`
- `Date of birth`
- `Created_date`

STUDENT (Subtype of USER):

- `Program`

COMMUNITY ADMIN (Subtype of USER):

- `Privileges`

SYSTEM ADMIN (Regular entity):

- `SysadminID` (primary key)
- `SystemAdminName`

UNIVERSITY (Regular entity):

- `Uni_ID` (primary key)

- Uni_name
- Uni_description
- Email_Domain

COMMUNITY (Regular entity):

- ID (primary key)
- Com_name
- Creation_date
- Com_description
- Number of participants (derived attribute)
- Number of community admins (derived attribute)

FORUM (Regular entity):

- ID (primary key)
- ForumName
- Description
- Creation_date

POST (Regular entity):

- ID (primary key)
- Content
- Created time
- Attachment (multivalued attribute)

COMMENT (Regular entity):

- ID (primary key)

- Content
- Created date

NOTIFICATION (Regular entity):

- ID (primary key)
- Content
- Noti-date
- Read_status

TAG (Regular entity):

- TagID (primary key)
- Tag Name
- Description

DISCUSSION ITEM (Category entity type):

- This category is formed from Forum, Post, and Comment; it has no intrinsic attributes in the EERD.

POST EDIT (Weak entity):

- No. (partial key)
- Old content
- Edit time

COMMENT EDIT (Weak entity):

- No. (partial key)
- Old content
- Edit time

1.2.2 Relations

The EERD contains 19 relationships. The maximum cardinalities shown in the diagram are described below.

- **MEMBER OF:** `STUDENT` (M) – `COMMUNITY` (N). A student may be a member of multiple communities, and a community may have multiple student members.
- **CONTROLS:** `COMMUNITY` (1) – `COMMUNITY ADMIN` (N). A community can be controlled by multiple community administrators; each community administrator controls one community.
- **HAS (Community–Forum):** `COMMUNITY` (1) – `FORUM` (N). A community can contain multiple forums, while each forum belongs to one community.
- **MANAGES:** `STUDENT` (N) – `FORUM` (M). A student may manage multiple forums, and a forum may be managed by multiple students.
- **HAS (Forum–Post):** `FORUM` (1) – `POST` (N). A forum can contain multiple posts, while each post belongs to one forum.
- **CREATES (Student–Post):** `STUDENT` (1) – `POST` (N). A student can create multiple posts, while each post is created by one student.
- **HAS (Post–Comment):** `POST` (1) – `COMMENT` (N). A post can have multiple comments, while each comment belongs to one post.
- **CREATES (Student–Comment):** `STUDENT` (1) – `COMMENT` (N). A student can create multiple comments, while each comment is created by one student.
- **PARENT (recursive):** `COMMENT` (Parent: 1) – `COMMENT` (Child: N). A parent comment can have multiple child comments, and each child comment has one parent comment.
- **ATTACHES:** `POST` (N) – `TAG` (M). A post can be associated with multiple tags, and a tag can be attached to multiple posts.

- **VOTES:** STUDENT (N) – DISCUSSION ITEM (M). A student can vote on multiple discussion items, and a discussion item can receive votes from multiple students. This relationship has the attributes Upvote (Boolean) and Last_vote_date.
- **SUBSCRIBES NOTIFICATION TO:** USER (N) – DISCUSSION ITEM (M). A user can subscribe to notifications for multiple discussion items, and a discussion item can have multiple subscribing users.
- **RECEIVES:** USER (1) – NOTIFICATION (N). A user can receive multiple notifications, while each notification is received by one user.
- **ABOUT:** NOTIFICATION (N) – DISCUSSION ITEM (1). A discussion item can be the subject of multiple notifications, while each notification is about one discussion item.
- **STUDIES AT:** STUDENT (N) – UNIVERSITY (1). A university can have multiple students, while each student studies at one university.
- **HANDLES:** SYSTEM ADMIN (1) – UNIVERSITY (N). A system administrator can handle multiple universities, while each university is handled by one system administrator.
- **CORRESPONDS:** UNIVERSITY (1) – COMMUNITY (1). Each university corresponds to one community, and each community corresponds to one university.
- **RECORDS (identifying):** POST (1) – POST EDIT (N). A post can have multiple edit records; each Post Edit is identified through one post.
- **LOGS (identifying):** COMMENT (1) – COMMENT EDIT (N). A comment can have multiple edit records; each Comment Edit is identified through one comment.

Specialization and category constraints:

- STUDENT and COMMUNITY ADMIN are overlapping subtypes of USER, as indicated by the o specialization circle.

- DISCUSSION ITEM is a union/category formed from FORUM, POST, and COMMENT, as indicated by the u category circle.

1.3 Mapping from EERD to relational schema

PHỤ LỤC